Discovering Underlying Topics in the Newsgroups Dataset with Clustering and Topic Modeling

Subhendu Mandal
Assistant Professor at Centre for AI & ML

ITER — SOA University
Bhubaneswar, Odisha

January 1, 2026

Topics to Be Covered

- What is Unsupervised Learning?
- Types of Unsupervised Learning
- K-Means Clustering and its Working
- Implementing K-Means from Scratch
- K-Means using Scikit-learn
- Optimizing K-Means Clustering Models
- Term Frequency–Inverse Document Frequency (TF–IDF)
- Clustering Newsgroups Data using K-Means
- What is Topic Modeling?
- Non-negative Matrix Factorization (NMF)
- Latent Dirichlet Allocation (LDA)
- Topic Modeling on Newsgroups Data

Unsupervised Learning

Definition

- Unsupervised learning is a machine learning approach that works with **unlabeled data**. Unlike supervised learning, there is **no teacher** providing correct outputs.

Objective

- Identify **hidden patterns, structures, or commonalities** in the input data.
- Absence of ground truth means there is **no clear notion of correct or incorrect results**.

Example

- Dimensionality reduction techniques such as **t-SNE** are examples of unsupervised learning.
- These methods help **visualize and explore high-dimensional data**.

Motivation and Applications

When to Use Unsupervised Learning

- When data is unlabeled
- When manual labeling is expensive or impractical
- When the goal is exploratory data analysis

Importance in Natural Language Processing

- Text data is difficult to label accurately
- Labeling is often time-consuming and subjective
- Unsupervised methods help organize and analyze large text collections

Types of Unsupervised Learning

- **Clustering:** Groups data points based on similarity to reveal natural patterns. Common examples include customer segmentation and document grouping.
- **Association Rule Learning:** Discovers interesting relationships or co-occurrence patterns among features. Widely used in market basket analysis.
- **Anomaly (Outlier) Detection:** Identifies rare or unusual observations that deviate significantly from normal data patterns. Useful in fraud detection and fault monitoring.
- **Projection (Dimensionality Reduction):** Transforms high-dimensional data into a lower-dimensional space while preserving essential structure and information. Examples include PCA, t-SNE.

Clustering Newsgroups Data using K-Means

The newsgroups dataset contains documents labeled with different topic categories. Some categories are closely related or even overlapping. Examples include:

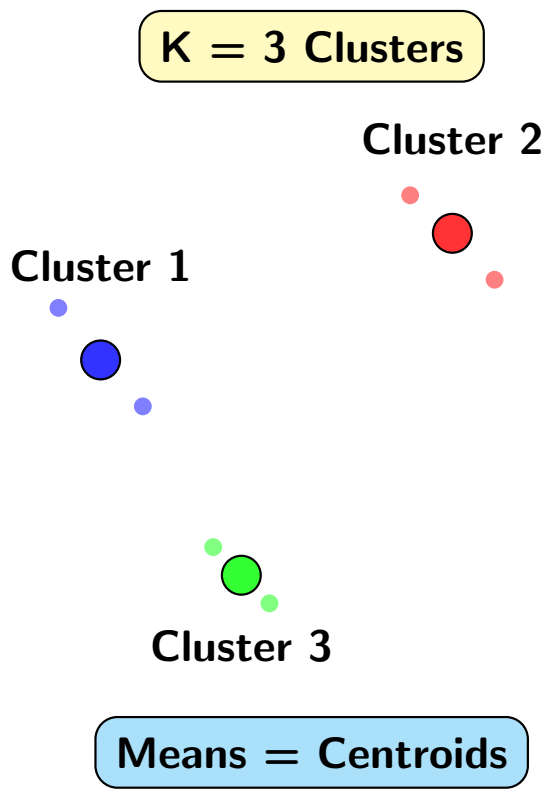
- Computer-related groups:
  - comp.graphics
  - comp.os.ms-windows.misc
  - comp.sys.ibm.pc.hardware
  - comp.sys.mac.hardware
  - comp.windows.x
- Religion-related groups:
  - alt.atheism
  - talk.religion.misc

In this study, we assume that the labels are unknown and apply **k-means clustering** to examine whether related topics are grouped together automatically.

K-Means Clustering: K and Means

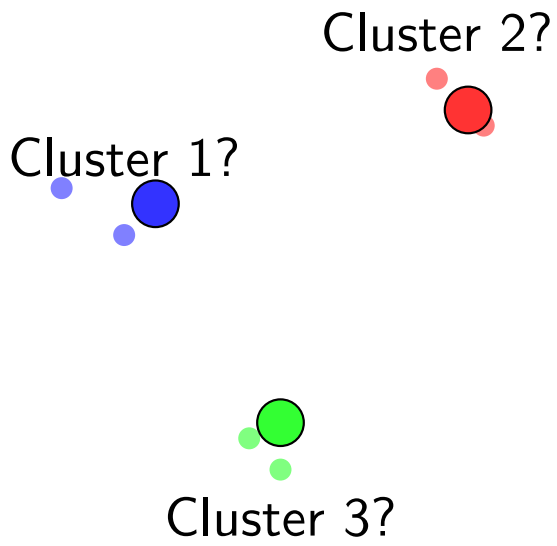
Definition:

K-means is an unsupervised algorithm that partitions data into **k clusters** based on feature similarity. Each cluster has a **centroid**, and each point belongs to the nearest centroid.



Steps in the K-Means Algorithm

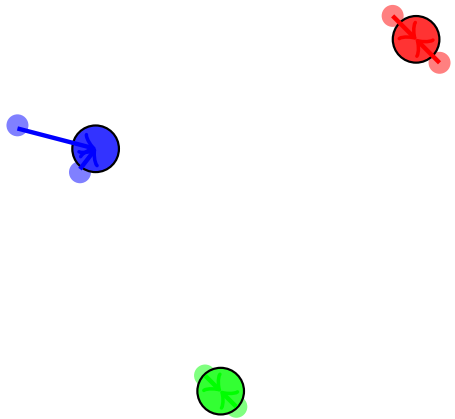
Step 1:The algorithm starts with randomly selecting k (predefined) samples from the dataset as centroids.



Steps in the K-Means Algorithm

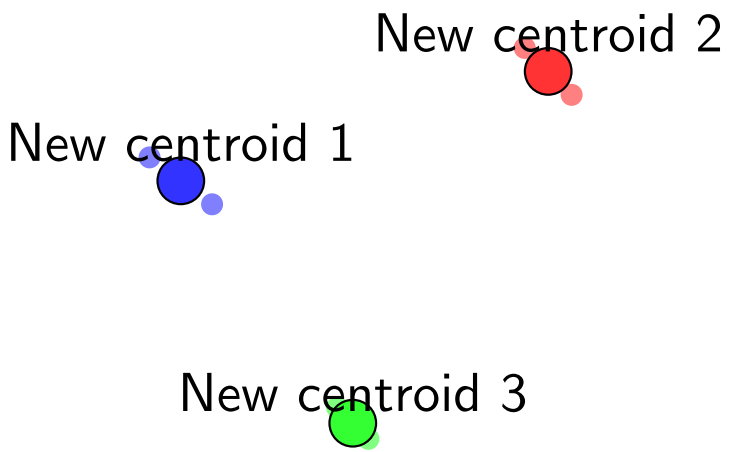
Step 2:Assign Clusters

- Assign each sample to the nearest centroid
- Distance is commonly measured using **Euclidean distance**.
- Other metrics can also be used, such as the **Manhattan distance** and **Chebyshev distance**,



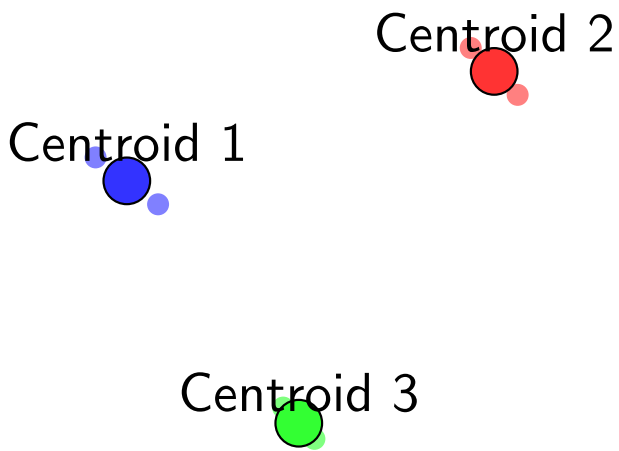
Steps in the K-Means Algorithm

Step 3: Recalculate the centroids as the mean of points in each cluster.



Steps in the K-Means Algorithm

Step 4: Repeat assignment and update steps until centroids stop moving significantly.



Distance Metrics Used in K-Means

Consider two points in 2D: (x_1, y_1) and (x_2, y_2).

Euclidean Distance

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Manhattan Distance

$$d = |x_2 - x_1| + |y_2 - y_1|$$

Chebyshev Distance

$$d = \max\left(|x_2 - x_1|, |y_2 - y_1|\right)$$

K-Means with scikit-learn: Step 1

Import and Initialize the KMeans Model

```
from sklearn.cluster import KMeans
kmeans_sk = KMeans(n_clusters=3, random_state=42)
```

Important Parameters:

KMeans Parameters

Parameter	Default	Example	Description
n_clusters	8	3,5,10	Number of clusters K
max_iter	300	10,100,500	Max iterations
tol	1e-4	1e-5,1e-8	Convergence tolerance
random_state	None	0,42	Random seed

K-Means with scikit-learn: Step 2

Fit the Model on Data and Obtain Results

```
1 kmeans_sk.fit(X)
2
3 clusters_sk = kmeans_sk.labels_
4 centroids_sk = kmeans_sk.cluster_centers_
```

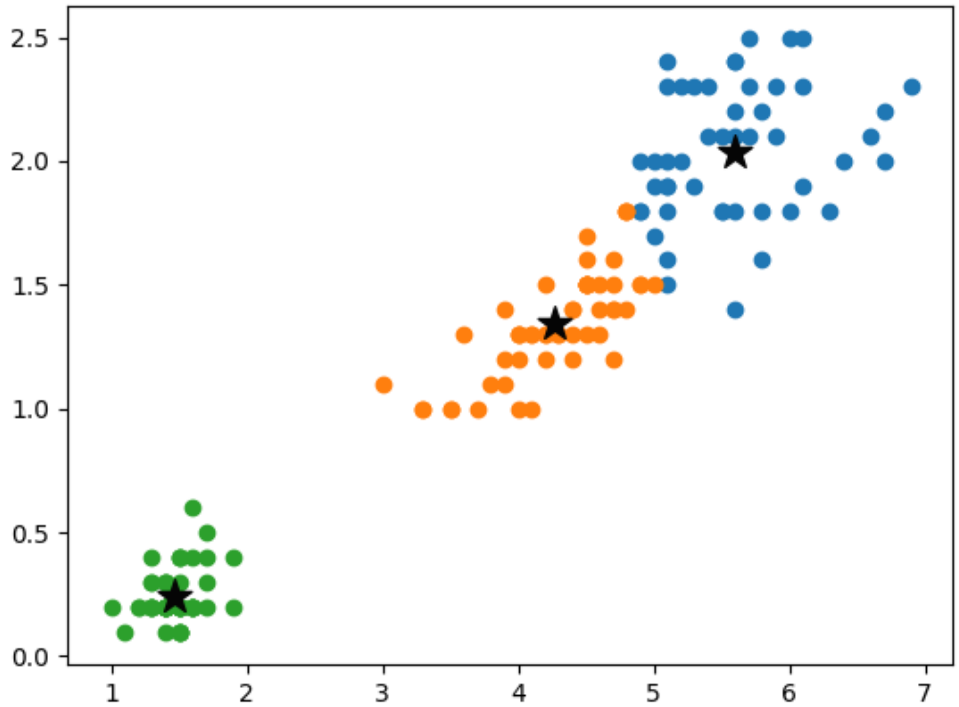
Cluster Results

- **clusters_sk:** Array of cluster assignments for each data point
- **centroids_sk:** Coordinates of the K cluster centroids

```
plt.scatter(X[:, 0], X[:, 1], c=clusters_sk)
plt.scatter(centroids_sk[:, 0], centroids_sk[:, 1],
            marker='*', s=200, c='#050505')
plt.show()
```

Plot Result

- Data points are colored by cluster
- Centroids are marked as stars
- Results visually match custom K-Means implementation



Data samples along with learned cluster centroids

Choosing the Value of k (Elbow Method)

Problem

In most clustering tasks, we do not know the optimal number of clusters (k). K-means requires k as an input, so we need a systematic way to choose it.

Elbow Method

- Train K-means models with different values of k.
- For each k, compute the sum of squared errors (SSE), also called within-cluster sum of squares.
- Plot SSE vs. k and look for the "elbow" point where the SSE decrease slows down.
- The value at the elbow is considered the optimal k.

Sum of Squared Errors (SSE)

SSE Formula

$$SSE = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

$$= \underbrace{\|x_1 - \mu_1\|^2 + \|x_2 - \mu_1\|^2 + \dots}_{\text{Cluster 1}}$$

$$+ \underbrace{\|x_3 - \mu_2\|^2 + \dots}_{\text{Cluster 2}} + \dots$$

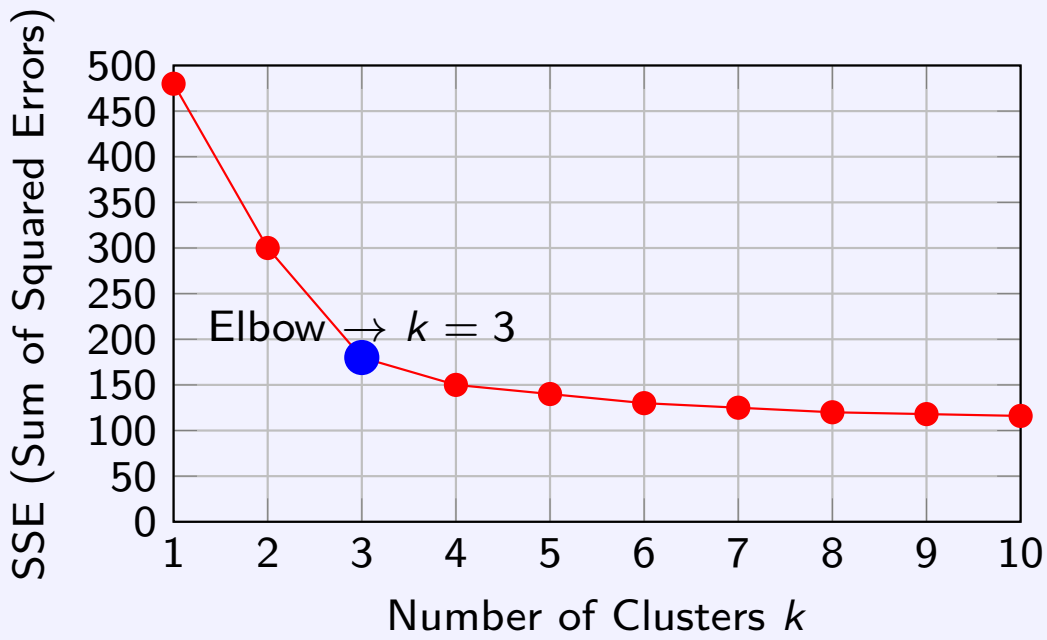
Interpretation

Step-by-step:

- For each cluster, calculate the squared distance of each point to its centroid.
- Sum these squared distances for all points in that cluster.
- Sum over all clusters to get total SSE.
- Plot SSE vs k to find the "elbow" (optimal k).

Elbow Method: Choosing Optimal k

Elbow Curve Plot



Simple Explanation

The elbow method finds the point where WCSS stops dropping quickly. Before this point, adding clusters gives big improvement. After this point, the improvement becomes very small.

Numerical Example of K-Means Clustering

Dataset

$$A_1(1,9),\ A_2(2,6),\ A_3(7,5),\ A_4(6,8),$$
$$A_5(8,6),\ A_6(5,4),\ A_7(1,3),\ A_8(4,7)$$

Initial Cluster Centroids

$$C_1 = A_1(1,9), \quad C_2 = A_4(6,8), \quad C_3 = A_7(1,3)$$

Distance Metric

$$d(a,b) = |x_1 - x_2| + |y_1 - y_2|$$

Using the K-Means clustering algorithm, perform the clustering process and determine the updated cluster centroids after the first two iterations. Clearly show all intermediate calculations.

Iteration 1: Distance Computation and Cluster Assignment

Point	d to C_1	d to C_2	d to C_3	Cluster
$A_1(1,9)$	0	6	6	C_1
$A_2(2,6)$	4	6	4	C_1
$A_3(7,5)$	10	4	8	C_2
$A_4(6,8)$	6	0	10	C_2
$A_5(8,6)$	10	4	10	C_2
$A_6(5,4)$	9	5	5	C_2
$A_7(1,3)$	6	10	0	C_3
$A_8(4,7)$	5	3	7	C_2

Clusters after Iteration 1

$$C_1 : \{A_1, A_2\}$$
$$C_2 : \{A_3, A_4, A_5, A_6, A_8\}$$
$$C_3 : \{A_7\}$$

Updated Centroids after Iteration 1

$$C_1^{(new)} = \left(\frac{1+2}{2}, \frac{9+6}{2}\right) = (1.5, 7.5)$$
$$C_2^{(new)} = \left(\frac{7+6+8+5+4}{5}, \frac{5+8+6+4+7}{5}\right) = (6, 6)$$
$$C_3^{(new)} = (1, 3)$$

Iteration 2: Distance Computation and Cluster Assignment

Point	d to $C_1^{(new)}$	d to $C_2^{(new)}$	d to C_3	Cluster
$A_1(1,9)$	2	8	6	C_1
$A_2(2,6)$	2	4	4	C_1
$A_3(7,5)$	8	2	8	C_2
$A_4(6,8)$	5	2	10	C_2
$A_5(8,6)$	8	2	10	C_2
$A_6(5,4)$	7	3	5	C_2
$A_7(1,3)$	5	7	0	C_3
$A_8(4,7)$	3	3	7	C_1

Clusters after Iteration 2

$$C_1 : \{A_1, A_2, A_8\}$$
$$C_2 : \{A_3, A_4, A_5, A_6\}$$
$$C_3 : \{A_7\}$$

Updated Centroids after Iteration 2

$$C_1^{(new)} = \left(\frac{1+2+4}{3}, \frac{9+6+7}{3}\right) = (2.33, 7.33)$$
$$C_2^{(new)} = \left(\frac{7+6+8+5}{4}, \frac{5+8+6+4}{4}\right) = (6.5, 5.75)$$
$$C_3^{(new)} = (1, 3)$$

Final Answer

The updated cluster centroids after the first two iterations are:

$$C_1 = (2.33, 7.33), \quad C_2 = (6.5, 5.75), \quad C_3 = (1, 3)$$

Exercise: K-Means Clustering Using Euclidean Distance

Dataset

$$A_1(1,9),\ A_2(2,6),\ A_3(7,5),\ A_4(6,8),$$
$$A_5(8,6),\ A_6(5,4),\ A_7(1,3),\ A_8(4,7)$$

Initial Cluster Centroids

$$C_1 = A_1(1,9), \quad C_2 = A_4(6,8), \quad C_3 = A_7(1,3)$$

Distance Metric (Euclidean Distance)

$$d(a,b) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Using the K-Means clustering algorithm and the Euclidean distance metric, perform the clustering process and determine the updated cluster centroids after the first two iterations. Clearly show all intermediate calculations.

Implement K-Means Clustering from Scratch Using Python

- Generate 100 random data points
- Cluster the data into 3 clusters
- Understand the algorithm step-by-step

Generating 100 Data Points

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 np.random.seed(42)
5
6 # Generate 100 random 2D points
7 X = np.random.rand(100, 2)
```

Implement K-Means Clustering from Scratch Using Python

Initialize 3 Clusters
Select Initial Centroids

```
K = 3  # Number of clusters

# Randomly select K data points as initial centroids
centroids = X[np.random.choice(len(X), K, replace=False)]
```

Compute Euclidean Distance

```
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((point1 - point2) ** 2))
```

Implement K-Means Clustering from Scratch Using Python

Assign Each Point to a Cluster

```
def assign_clusters(X, centroids):
    clusters = []
    for x in X:
        distances = [
            euclidean_distance(x, c)
            for c in centroids
        ]
        clusters.append(np.argmin(distances))
    return np.array(clusters)
```

Implement K-Means Clustering from Scratch Using Python

Update Cluster Centroids

```
1  def update_centroids(X, clusters, K):
2      new_centroids = []
3
4      for k in range(K):
5          cluster_points = X[clusters == k]
6          new_centroids.append(
7              cluster_points.mean(axis=0)
8          )
9
10     return np.array(new_centroids)
```

Implement K-Means Clustering from Scratch Using Python

Cluster All Data Points(K-Means Iterative Process)

```
max_iterations = 100

for _ in range(max_iterations):
    clusters = assign_clusters(X, centroids)
    new_centroids = update_centroids(X, clusters, K)

    if np.all(centroids == new_centroids):
        break

    centroids = new_centroids
```

Display the final Centroids positions

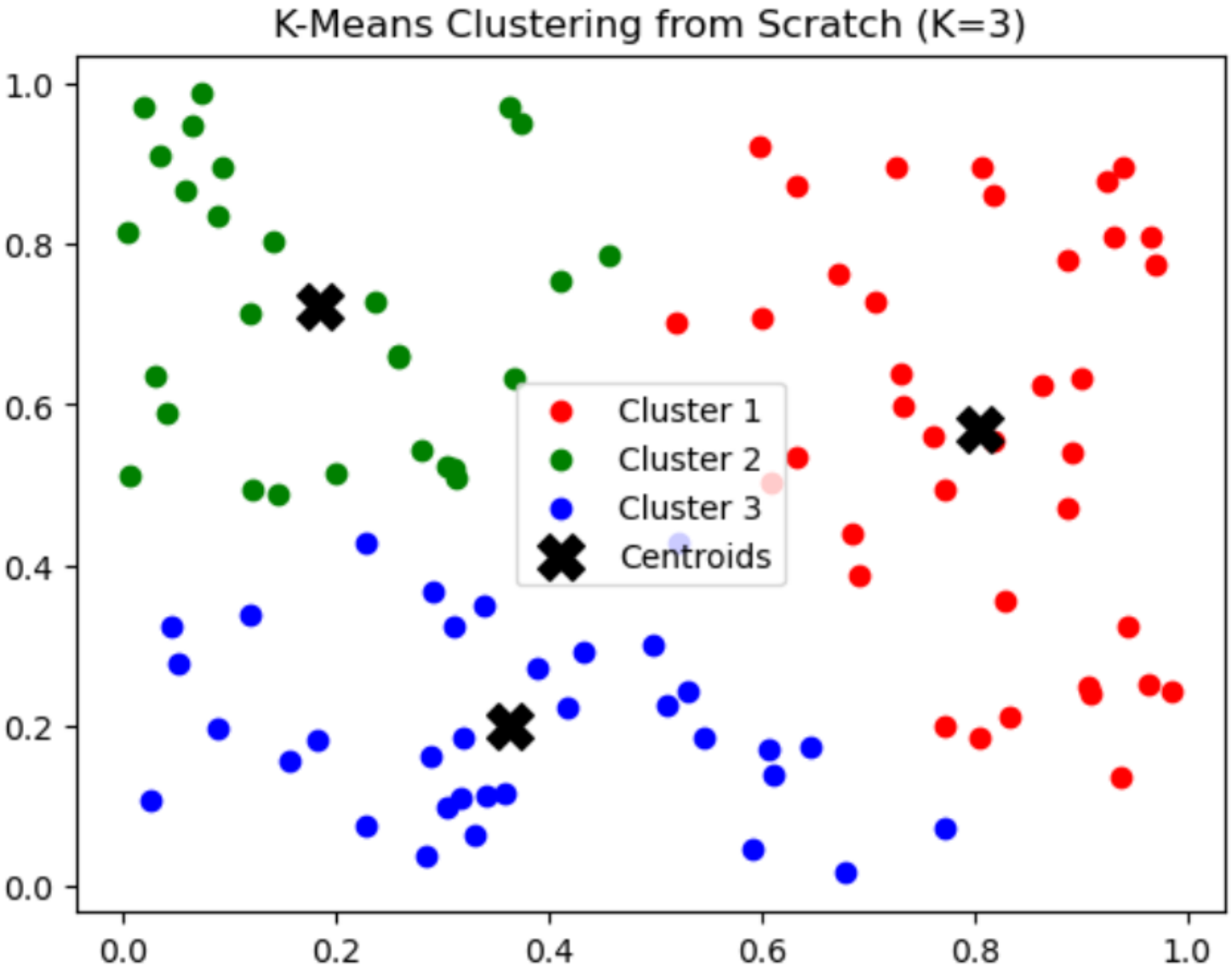
```
    print(centroids)
```

Implement K-Means Clustering from Scratch Using Python

Visualization of Clusters and Centroids

```
colors = ['red', 'green', 'blue']
for i, cluster in enumerate(clusters):
    cluster = np.array(cluster)
    plt.scatter(
        cluster[:, 0],
        cluster[:, 1],
        c=colors[i],
        label=f'Cluster {i+1}'
    )
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    c='black',
    marker='X',
    s=200,
    label='Centroids')
plt.legend()
plt.title("K-Means Clustering from Scratch (K=3)")
plt.show()
```

Visualization of Clusters and Centroids



Why Density-Based Clustering?

- Discovers clusters of **arbitrary shape**
- Does not require number of clusters in advance
- Robust to **noise and outliers**
- Well suited for spatial data

DBSCAN = Density-Based Spatial Clustering of Applications with Noise

Key Parameters in DBSCAN

1. ε (Epsilon)

Radius of the neighborhood around a point

2. MinPts

Minimum number of points required to form a dense region

Types of Points in DBSCAN

Core Point

A point having at least **MinPts** within its ε-neighborhood

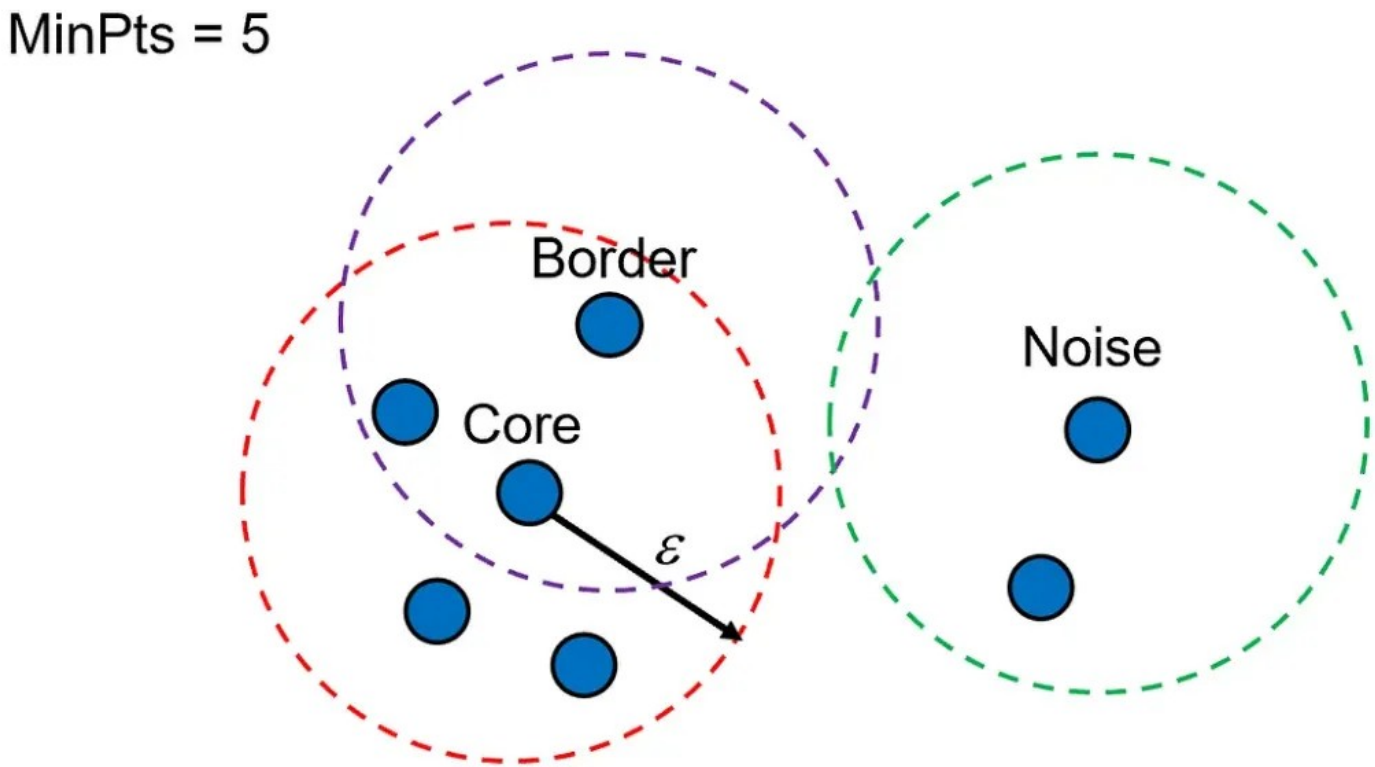
Border Point

A point that is within ε of a core point but has fewer than MinPts neighbors

Noise Point

A point that is neither a core point nor a border point

Visual Illustration: Core, Border, and Noise Points



Density-Reachable and Density-Connected Points

Density-Reachable

A point q is density-reachable from point p if:

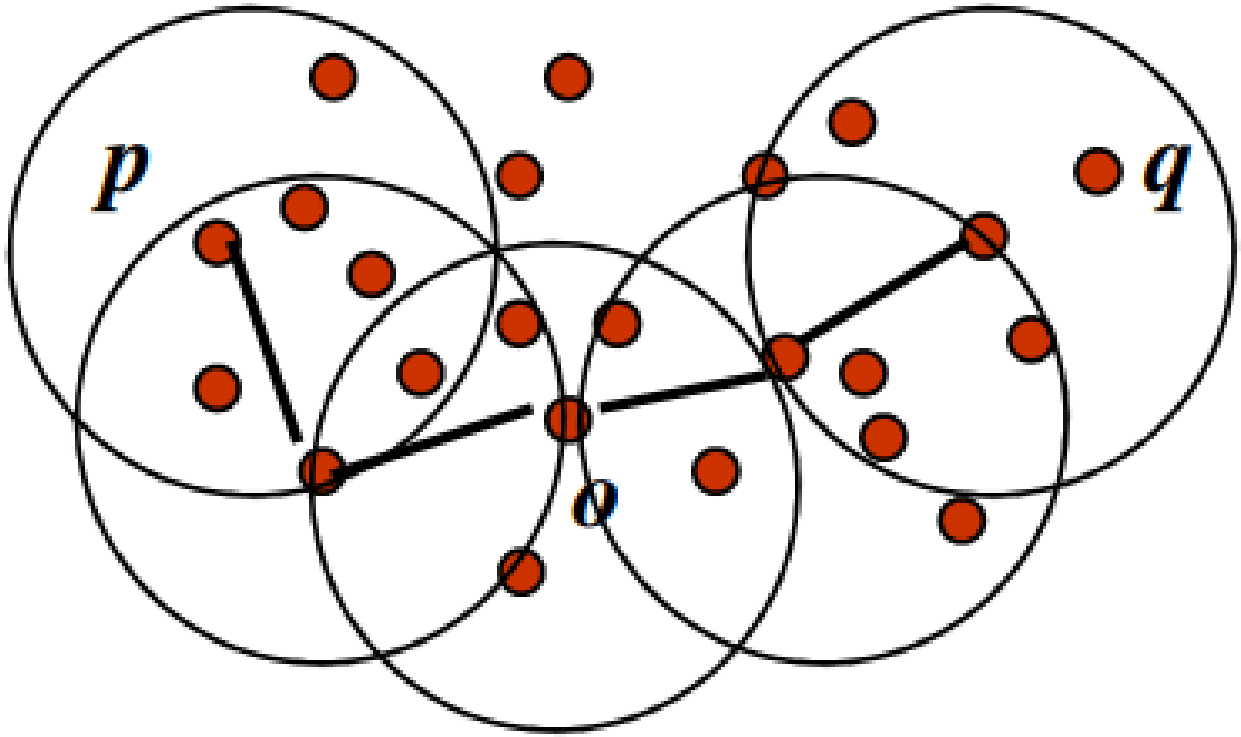
- q lies within ε of p
- p is a core point

Density-Connected

Two points p and q are density-connected if:

- There exists a point o such that both p and q are density-reachable from o

Visual Illustration: Density Connectivity



Here p and q are density connected through a sequence of points.

Steps in the DBSCAN Algorithm

1. **Identify Core Points** For each data point, count the number of points within its ε-neighborhood. If the number of points is greater than or equal to **MinPts**, the point is marked as a **core point**.
2. **Form Clusters** For each core point that has not yet been assigned to any cluster:
   - Create a new cluster
   - Recursively add all points that are **density-reachable** from this core point
3. **Density Connectivity** Two points *a* and *b* are said to be **density-connected** if there exists a sequence of points a = P1, P2,..., Pn = b such that:
   - Each consecutive pair of points lies within the ε-neighborhood
   - At least one point in the sequence is a **core point**

   This chaining process ensures that all points in a cluster are connected through dense regions.
4. **Label Noise Points** After all core points and their density-connected points are processed, any remaining unassigned points are labeled as **noise (outliers)**.

Summary of DBSCAN

Advantages

- Finds clusters of arbitrary shape
- Automatically detects noise
- No need to specify number of clusters

Limitations

- Sensitive to choice of ε and MinPts
- Struggles with varying densities